01_CODE_ALONG_LAB

Pintop | 2026 Training

CLASSROOM CODE-ALONG - Password Handling & Token-Session Design (Backend Design & Engineering)

Pintop Technologies | Class 07 | Instructor and student companion

What we will produce

Recreate a documented excerpt from the **real project** (`app/Http/Controllers/PayTrackAuthController.php`), practise a small safe variation, verify using the complete app or appropriate tool, and record both success and failure evidence. Preserve the original project.

Why: Your database is copied by an attacker. If passwords are stored as typed, every account is theirs. Today we learn to store passwords so that even a leak does not hand over the accounts, and to keep users logged in safely.

1. Find and read the real file

`codebase/app/Http/Controllers/PayTrackAuthController.php`

The excerpt is in `REFERENCE EXCERPT.php`. Its job is to give you something to type, trace and understand. It is not guaranteed to run on its own: it is part of a larger file. Do not overwrite project source with a fragment.

2. Set up the real project

```
cd "Backend Design & Engineering/codebase"
composer install
cp .env.example .env
php artisan key:generate
php artisan migrate --seed
php artisan serve --port=8000
```

3. Type in stages

Stage 1

Say: "I am writing a small section of the actual project. Look for the inputs and names; we will connect it with the next block."

```
use Illuminate\Support\Str;
class PayTrackAuthController {
 private function issue(int $userId):array {
  $access=Str::random(64);$refresh=Str::random(64);
  DB::table('api_tokens')->insert(['user_id'=>$userId,'token_hash'=>hash('sha256',$access),'expires_at'=>no
  DB::table('refresh_tokens')->insert(['user_id'=>$userId,'token_hash'=>hash('sha256',$refresh),'expires_at
  return ['access_token'=>$access,'expires_in'=>900,'refresh_token'=>$refresh];
 }
 public function login(Request $r){
```

Stage 2

Say: "I am writing a small section of the actual project. Look for the inputs and names; we will connect it with the next block."

```
        $data=$r->validate(['email'=>'required|email','password'=>'required|string']);
        $user=DB::table('users')->where('email',strtolower($data['email']))->first();
        if (!$user||!Hash::check($data['password'],$user->password))return response()->json(['message'=>'Invalid
        return response()->json($this->issue((int)$user->id));
       }
       public function refresh(Request $r){
        $data=$r->validate(['refresh_token'=>'required|string']);
        try{$fresh=DB::transaction(function()use($data){
         $row=DB::table('refresh_tokens')->where('token_hash',hash('sha256',$data['refresh_token']))->lockForUpda
```

Stage 3

Say: "I am writing a small section of the actual project. Look for the inputs and names; we will connect it with the next block."

```
         if (!$row||$row->revoked_at||\Carbon\Carbon::parse($row->expires_at)->isPast())return null;
         DB::table('refresh_tokens')->where('id',$row->id)->update(['revoked_at'=>now(),'updated_at'=>now()]);
         return $this->issue((int)$row->user_id);
        },3);
        return $fresh?response()->json($fresh):response()->json(['message'=>'Invalid refresh token'],401);
        }catch(\Throwable $e){report($e);return response()->json(['message'=>'Refresh not available'],503);}
       }
      }
```

4. Complete these activities

1. **Trace the input from the login or token request to the verification routine.**
2. **Write down a failed authentication case and its expected status; do not display real secrets.**
3. **Add or adjust one local-only assertion, and repeat the success and failure case.**

5. Check, debug and record

```
      php artisan route:list --path=api
      php artisan test
```

Expected check: If the local app starts or its tests pass, capture evidence of the specific behaviour tied to this class, not merely an application startup. Test one failed or edge input too. Log the exact error if blocked.

6. Marking

- 4/20: explain concept and input/output in plain language
- 4/20: accurate typed or edited work
- 4/20: local proof and explanation
- 4/20: safe variation and wrong-case recovery
- 4/20: short documented submission

Lab ethics: no unapproved public scanning, no third-party tokens and no real banking customer data.